CÓDIGO LIMPIO
NOMBRES Y ESTRUCTURA

Buenas prácticas basadas en Robert C. Martin (Uncle Bob)

¿QUÉ ES CÓDIGO LIMPIO?

Se lee mucho más de lo que se escribe

El código se lee muchas más veces de las que se escribe. Por eso Robert C. Martin sostiene que la legibilidad no es un lujo: es la característica más importante de un buen código.

Código limpio es aquel que cualquier otra persona (o vos mismo, seis meses después) puede leer, entender y modificar sin tener que adivinar qué hace cada cosa.

La regla del Boy Scout resume la actitud: dejá el código un poco mejor de como lo encontraste, aunque el cambio no te lo hayan pedido a vos.

CONFUSOS vs CLAROS

NOMBRES CONFUSOS

- **Nombres de una letra**

int d; — ¿qué es d? Nadie lo sabe sin leer todo el método de nuevo.

- **Abreviaturas ambiguas**

genymdhms() mezcla generar, año, mes, día, hora, minuto y segundo en un solo nombre impronunciable.

- **El mismo nombre, dos sentidos**

Usar info y data como sinónimos intercambiables dentro de la misma clase, sin ninguna diferencia real.

NOMBRES CLAROS

- **Nombres que revelan intención**

int elapsedTimeInDays; — el nombre solo ya explica qué guarda la variable.

- **Una palabra por concepto**

Elegir fetch, retrieve o get para la misma operación en cada clase evita dudas al leer el código.

- **Pronunciables y buscables**

generationTimestamp se puede decir en voz alta y buscar en todo el proyecto sin ambigüedad.

REGLAS PARA NOMBRAR (I)

Revela la intención

El nombre debe decir por qué existe, qué hace y cómo se usa. Si necesita un comentario al lado, no cumplió su función.

Evitar desinformación

No uses palabras con un significado ya establecido y distinto: accountList si en realidad no es una List.

Distinciones significativas

Diferenciar nombres agregando números o palabras vacías (moneyAmount vs money) no aporta información real.

Nombres pronunciables

Si no podés decir el nombre en voz alta sin trabarte, va a ser difícil discutirlo en una reunión.

REGLAS PARA NOMBRAR (II)

Nombres buscables

Una letra o un número mágico son imposibles de buscar en todo el proyecto. Usá constantes con nombre.

Evitar mapeo mental

El lector no debería tener que traducir mentalmente tus nombres cortos a su significado real cada vez.

Un nombre por concepto

Elegí una sola palabra para una idea (por ejemplo, buscar) y usala siempre; no la mezcles con obtener o traer.

Contexto justo y necesario

Agrupar nombres relacionados en una clase les da contexto (Direccion.calle); repetir el prefijo en todos lados sobra.

Sustantivos, verbos y sin decoraciones

Las clases y sus atributos se nombran con sustantivos: Cliente, Pedido, fechaDeAlta. Un nombre de clase nunca debería ser un verbo.

Los métodos se nombran con verbos: calcularTotal(), registrarPrestamo(), esValido(). Los que consultan un atributo suelen empezar con get, tiene o es.

Evitá la notación húngara y los prefijos de miembro (m_nombre, strNombre): el compilador y el editor ya conocen el tipo y el alcance de cada variable.

NOMBRES EN LA BIBLIOTECA



```
public class Libro {
  private double d;
  private int cnt;
  private boolean flag;
  public boolean chk()
  public void proc(int x)
}
// ¿d? ¿cnt? ¿flag? ¿x?
```

```
public class Libro {
  private double duracionPrestamoDias;
  private int cantEjemplaresDisponibles;
  public boolean
    tieneEjemplaresDisponibles()
// cada nombre se explica solo
```

FUNCIONES: UNA SOLA COSA

Pequeñas, y de un solo nivel de abstracción

Las funciones deben ser pequeñas y hacer una sola cosa. Si un método mezcla validar, calcular y guardar, en realidad son tres funciones escondidas en una.

Todas las instrucciones de una función deberían estar en el mismo nivel de abstracción: mezclar el índice de un array con un concepto de alto nivel como procesarPedido() la hace difícil de seguir.

Leído de arriba hacia abajo, el código debería contarse como una historia: cada función llama a las que están un nivel más abajo, como los párrafos de un diario.

FUNCIONES LIMPIAS



Pocos argumentos

Ideal: cero o un argumento. A partir de tres, el lector hace un esfuerzo extra por recordar el orden y el sentido de cada uno.

Sin efectos secundarios

Una función debe hacer solo lo que su nombre promete. Si además modifica un estado oculto, el nombre está mintiendo.

Comando o consulta, no ambos

Una función responde una pregunta o ejecuta una acción, nunca las dos cosas a la vez.

Preferir excepciones

Devolver códigos de error obliga a revisarlos en cada llamado. Una excepción separa el error del flujo normal.

REFACTOR: Antes y Después



```java
public boolean
 extraer(double m, boolean f) {
    if (f) log(m);
    if (m > saldo) return false;
  saldo -= m;
  return true;
}
// ¿qué hace f? ¿y ese true/false?
```

```java
public void extraer(double monto)
  throws SaldoInsuficiente {
  if (monto > saldo)
    throw new SaldoInsuficiente();
  saldo -= monto;
}
// una sola cosa, sin flags
```

El código también se lee con los ojos

El orden vertical importa: las variables se declaran cerca de donde se usan, y los conceptos relacionados se agrupan sin líneas en blanco de por medio que los separen.

Un archivo se lee como un diario: los conceptos generales y las funciones de alto nivel van arriba; los detalles y las funciones auxiliares, más abajo, cerca de quien las usa por primera vez.

Las líneas cortas y los archivos y funciones chicas no son un capricho estético: reducen el esfuerzo real de leer y mantener el código.

CHECKLIST RÁPIDO

¿El nombre revela la intención?

Si necesitás un comentario para explicar qué hace una variable o función, probablemente el nombre esté mal elegido.

¿La función hace una sola cosa?

Si aparece un "y" al describirla (valida y guarda), es señal de que hay que dividirla en dos.

¿Pocos argumentos y sin flags?

Un booleano como parámetro casi siempre significa que la función hace dos cosas distintas según su valor.

¿Quedó mejor que como lo encontraste?

La regla del Boy Scout: no hace falta una reescritura completa, alcanza con dejarlo un poco más limpio en cada cambio.



MENSAJE FINAL

"El código limpio siempre parece que fue escrito por alguien a quien le importa."

Nombres claros y funciones pequeñas no son un adorno: son la forma más simple de que el código siga siendo entendible para quien lo lea después.